GOSIM®
GLOBAL OPEN-SOURCE INNOVATION MEETUP

GOSIM Shenzhen 2026

Topic

MemOS：Agent自主进化的下一代记忆系统

MemOS: The Next-Generation Memory System for Autonomous Agent Evolution

庄郭冕 Guomian Zhuang

记忆张量应用算法负责人

Head of Applied Algorithms, Memory Tensor (Shanghai) Technology Co., Ltd.

GOSIM

GOSIM Shenzhen 2026

October 16-17, 2026 Shenzhen, China

GOSIM

GOSIM Shenzhen 2026

MemOS：Agent 自主进化的下一代记忆系统

庄郭冕

记忆张量（上海）科技有限公司

CONTENTS

Part 01 记忆张量团队介绍

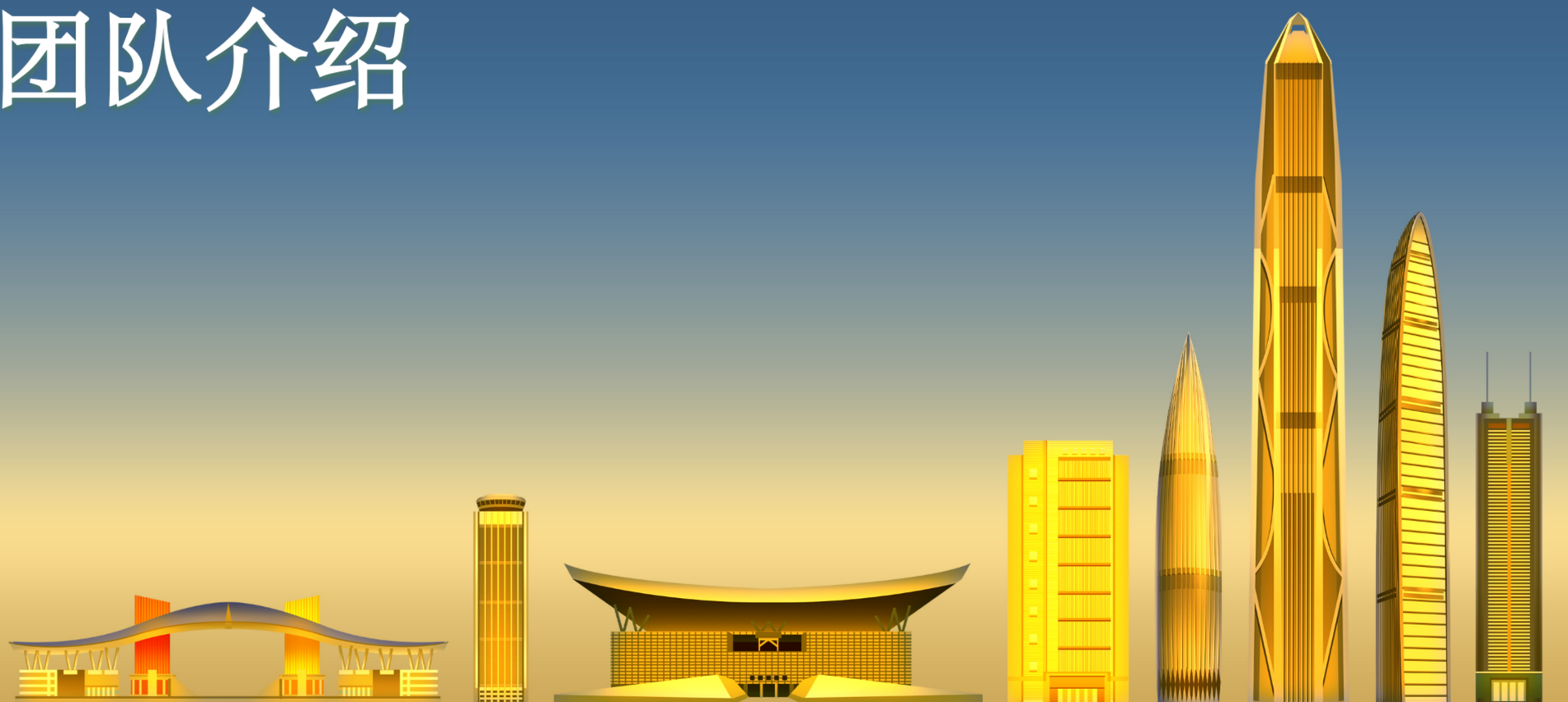
Part 02 MemOS 1.0: 让记忆可管理、可调度

Part 03 MemOS 2.0: 让记忆可随Agent执行进化

Part 04 下一代记忆系统: MemOS x Metis 的统一记忆架构

01

记忆张量团队介绍



记忆张量团队简介

驱动持续突破

复合型团队

由上海算法创新研究院孵化，鄂维南院士担任首席科学顾问，团队汇聚了数学、计算机领域学术大咖与互联网大厂精英，是国内极少数基于人工智能基础原理的迭代，且有能力驱动商业研发创新的大模型团队

专注架构创新

强研发实力

专注AI、应用数学30年+，200+ 顶会/刊论文，5万+引用次数，专注于人工智能基本原理的系统性创新， 致力于走出一条符合中国国情的全新大模型应用路线

低成本/低幻觉/高泛化

专业市场服务

团队主导过多种类型的超大规模商业化项目，致力于以“低成本、低幻觉、高泛化”的方式，提供高效实用的大模型服务，使人工智能带来的生产力进步红利惠及更广泛的行业和大众

Memory已成共识，记忆张量率先构建了完整的技术栈

Memory3 奠定记忆基础理论，让记忆可计算；MemOS 让记忆可治理与系统化；Metis 记忆原生模型让记忆成为模型持续成长的内生动力

01 Memory3 | 记忆分层

显性记忆 | 隐性记忆 | 明文记忆

2023

2.4B > 9B 模型

显式记忆压缩 1830倍，性能无损

02 MemOS | 首款记忆操作系统

文本 | 激活 | 参数化记忆统一调度

2025

Agent | User Memory

双线全球领先

云服务平台、本地化部署多版本支持

03 Metis | 记忆原生化

2026

持续交互/内生状态更新

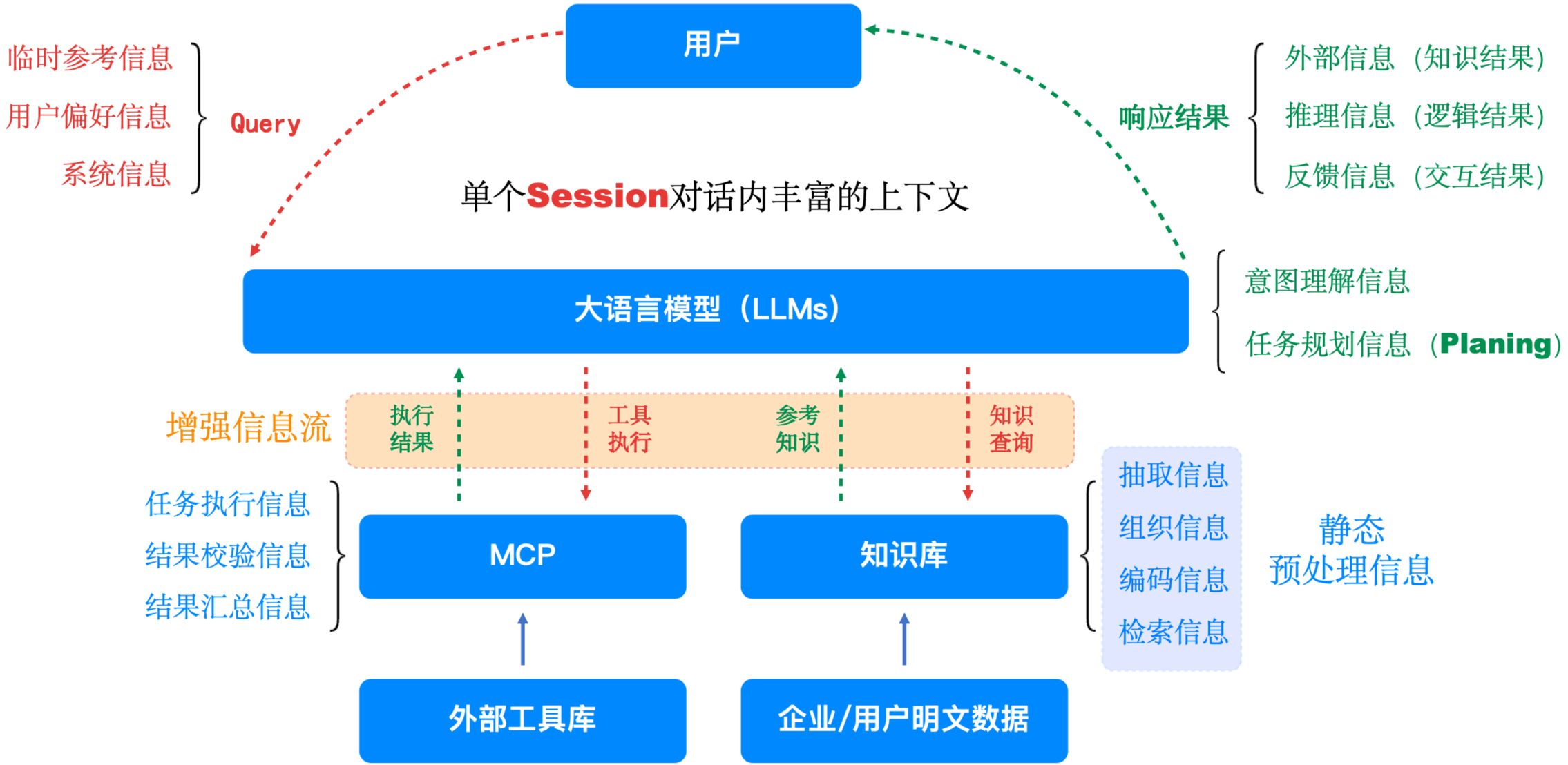
内化记忆能力持续迭代

参数决定模型出生时有多强；记忆决定模型在时间中能成长到多强

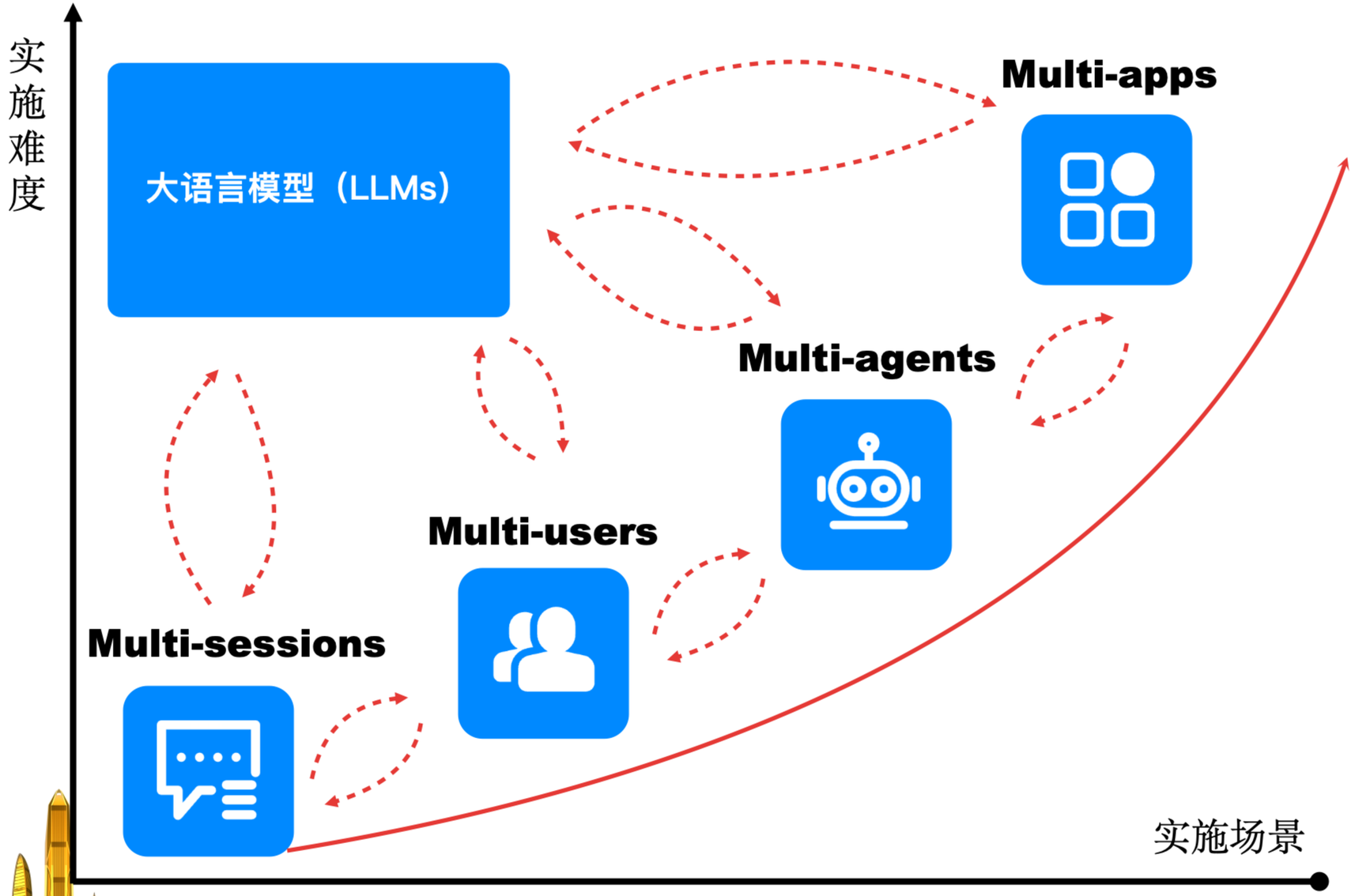
02

MemOS 1.0：让记忆可管理、可调度

从实践层面看记忆增强的必要性



从实践层面看记忆增强的必要性



应用开发
复杂度急剧增长

从实践层面看记忆管理框架的必要性

从实践层面看记忆管理框架的必要性

大模型记忆增强层的实现路径:(1)模型内生驱动的记忆增强

模型内生驱动的记忆增强：通过设计创新的基座模型架构，引入记忆增强的能力，强化模型的性能。

代表性工作	作者团队/时间	核心方案	技术特点
Memorizing Transformers	Google 2022	把局部上下文注意力和外部记忆检索融合，引入外部记忆	首次在语言模型中引入外部记忆联合模型解码
Focused Transformer	IDEAS NCBR 2023	引入对比训练，让 KV 空间更好区分上下文相关/无关信息	方法侧重训练策略改进，不改架构，可迁移到
MemoryLLM	UCSD 2024	在每层引入固定大小 memory tokens，作为可更新参数池	强调内置可更新记忆，能持续吸收新知识并抗遗忘
Memory3	记忆张量 2024	基于不同记忆类型进行分层管理和建模，缩减主干参数	首次提出记忆分层框架，对模型记忆进行分层建模
WISE	浙江大学 2024	提出双参数记忆：主记忆存预训练知识，侧记忆存编辑知识	面向 lifelong model editing进行记忆编辑
Titans	Google 2025	提出神经长时记忆模块，学习存储/遗忘	模拟人类记忆分层，支持超长上下文
MemAgent	ByteDance 2025	基于强化学习的Agents上下文外推扩充方案	侧重在短期记忆的扩充

大模型记忆增强层的实现路径:(2)应用外向驱动的记忆增强

模型外向驱动的记忆增强：通过设计Prompt / Agent 流，模拟记忆过程，增强模型性能

代表性框架	时间	Slogan	技术特点	Star数量
Letta /MemGPT	2023	Create stateful AI agents that truly remember, learn, and evolve.	开源社区驱动，目标构建具备状态记忆与自我改进能力的 LLM Agent 平台，非 Production-grade. （是最早受到计算机系统启发设计记忆框架的）	18.2k
Mem0	2024	Universal memory layer for AI Agents.	纯明文记忆管理框架，较早面向应用层提供记忆管理的代表性框架。服务生产环境，强调平台化服务。	39.2k
Zep	2024	Build Agents That Recall What Matters.	强调采用 Temporal Knowledge Graph（时序知识图谱）结构来组织长期记忆	17.6k
Memobase	2025	Profile-Based Long-Term Memory for AI Applications.	强调用户画像与事件时间线的记忆系统，将会话内容抽取为结构化 profile 和事件，并关联时间戳，形成用户长期记忆	2.1k
Memories.ai	2025	Building AI to See and Remember	强调多模态记忆，把原始视频转化成可搜索、带上下文关联的数据库，支持SaaS服务	非开源
非常多的TOP学术团队			从 Memory面临的创新问题出发，提出了对应的解决方案	

大模型记忆增强层的实现路径对比

对比项	MemOS 融合范式	基于模型驱动	基于应用驱动
定位	内外记忆协同，形成系统级记忆一体化	从模型底层嵌入记忆机制，改变模型本体	在应用层叠加记忆系统，管理交互与任务信息
关注点	兼顾认知深度与应用广度	提升模型的认知与学习能力	提升系统的连续性与个性化体验
实现方式	分层协同+多触点调度	新型架构、新的训练策略、新的建模策略	明文存取与索引- 会话/任务级状态管理
优势	读写效率全局最优	记忆读取效率更高，性能上限高	**记忆写入效率更高、落地快，易扩展**
局限	设计难度高，开发+理论 双重要求	研发成本高，落地周期长	依赖底层模型，缺乏深层学习，幻觉严重
价值导向	构建系统级可持续演进的个性化记忆	推动 AI 基础能力演进	驱动应用生态与商业落地

模型驱动决定上限，应用驱动决定下限，需要从系统层面结合两者

记忆系统的核心功能点

不仅依赖更大的模型与干净的数据，更需要一套完整的 **记忆操作系统功能链路** —— 让 AI 能记住、能组织、能调用、能更新、还能共享。

从交互中捕获关键信息，形成记忆片段

对记忆进行建模，构建逻辑与时间关系

按需快速调用相关历史记忆，辅助推理与生成

动态修正或替换过时记忆，保持知识新鲜

跨任务、跨个体共享知识，实现知识复用

幻觉高发

幻觉高发

MemOS的系统框架

Context
Answer
记忆编解码层
MemDecoding
Query
记忆联合解码
MemOS服务形态
API服务
MCP服务
本地化部署服务
记忆应用层
Business Flow 业务流
MemScheduler （记忆调度系统）
内置参数记忆
外挂参数记忆
激活记忆
明文记忆
记忆调度层
长期记忆
部署任务流
记忆任务流
规则模版配置
主动预测算法
调度执行
消息队列
调度埋点
MemCube实例
MemLoader/Dumper
MemCube注册/存档
MemGovernance （记忆治理系统）
记忆治理层
生命周期管理
水印服务管理
权限控制管理
隐私控制管理
MemDB
行业记忆 Cube
专家记忆 Cube
场景记忆 Cube
用户记忆 Cube
....
Vector DB （向量数据库）
Graph DB （向量数据库）
记忆存储层
Purchase
Publish
MemStore （记忆市场）

应用开发层
Agent / Chat / Pipeline
MemOS（记忆）
MCP（工具）
Large Language Models 基座层

MemOS的系统框架

记忆存储层

MemCube： 可独立打包的最小记忆单元

MemStore： 可交易的记忆市场平台，扩展至skill

MemOS的系统框架

记忆治理层

HaluMem: 面向记忆系统的幻觉评估框架

MemControl: Agent驱动的记忆权限管理器

MemOS的系统框架

记忆调度层

MemTrigger： 可配置的多粒度调度触发模版

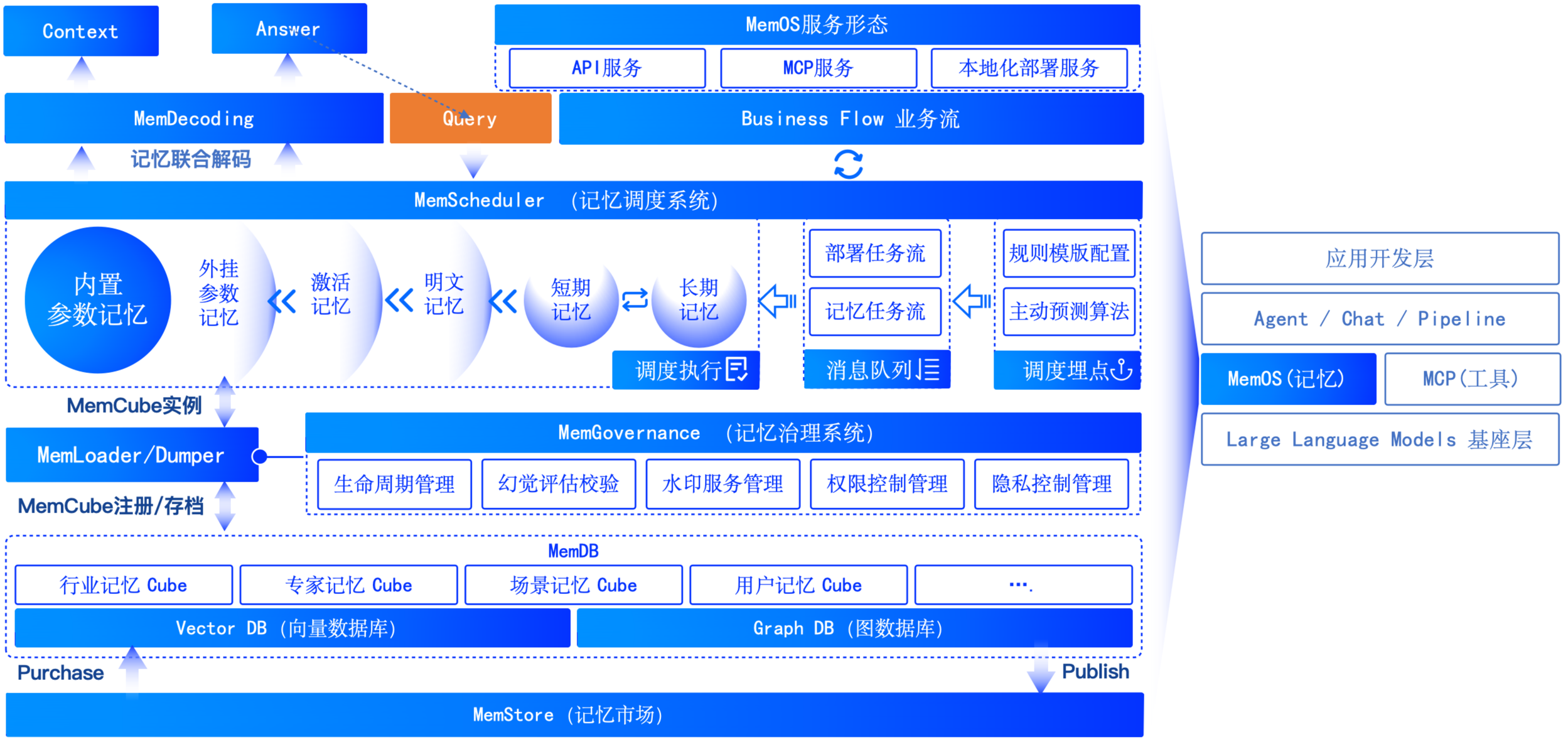
记忆分层： 融合场景的记忆分布建模机制

MemOS的系统框架

应用与编解码

- 应用开发层
- Agent / Chat / Pipeline
- MemOS（记忆）
- MCP（工具）
- Large Language Models 基座层

MemOS的系统框架


Context
Answer
MemOS服务形态
API服务
MCP服务
本地化部署服务
MemDecoding
Query
Business Flow 业务流
记忆联合解码
MemScheduler （记忆调度系统）
内置
参数记忆
外挂
参数
记忆
激活
记忆
明文
记忆
短期
记忆
长期
记忆
部署任务流
记忆任务流
规则模版配置
主动预测算法
调度执行
消息队列
调度埋点
MemCube实例
MemLoader/Dumper
MemGovernance （记忆治理系统）
生命周期管理
幻觉评估校验
水印服务管理
权限控制管理
隐私控制管理
MemCube注册/存档
MemDB
行业记忆 Cube
专家记忆 Cube
场景记忆 Cube
用户记忆 Cube
....
Vector DB （向量数据库）
Graph DB （图数据库）
Purchase
Publish
MemStore （记忆市场）
应用开发层
Agent / Chat / Pipeline
MemOS（记忆）
MCP（工具）
Large Language Models 基座层

GOSIM

GOSIM Shenzhen 2026

03

MemOS 2.0: 让记忆可随Agent执行进化

对话记忆 vs Agent 记忆：记忆对象的根本差异

不是"记忆更长了"，而是"记忆对象变了" — 后续所有算法设计均源于这组差异。

对话时代

多轮 · 问答

Round 1	用户提问 → 助手解释	纯文本 Q&A
Round 2	用户追问 → 助手补充、澄清、延展	上下文依赖前轮
Round 3	用户修正要求 → 助手继续回答	偏好修正

核心数据	主要难点	Memory目标
多轮 query / answer	上下文衔接与偏好延续	记住对话史和用户偏好

核心问题 如何"把话接住" — 服务于连贯表达

VS

Agent时代

多步 · 执行 · 反馈

query → context → 指令 → 执行 → 反馈结果 → 反思结果

下一轮指令

每一步迭代产生一个 grounded trace，循环直到任务完成或用户终止

patch /diff	bash /python	CLI /tool call	subagent	stdout /log	反思 / 偏好
代码修改	脚本执行	工具调用	子 Agent 协作	日志与观测	决策理由

+ 用户反馈：成功 · 追问 · 纠正

核心数据	主要难点	Memory目标
大量中间产物	组织、筛选、回填价值	把过程变成可复用能力

核心问题 如何"把任务跑通并记住为什么能跑通"
— 服务于长期协作与能力进化

核心区别

对话记忆 → 记“说过什么”； Agent 记忆 → 记"做过什么、为什么、哪里错了、如何纠正"。

Agent记忆最特别的地方

不是"记住更多对话"，而是记住任务执行的全过程和用户对结果的反馈— 这两者构成了记忆的价值来源

探索过程

Agent 的执行 = 多步结构化链路

读代码 → 改文件 → 报错 ✗ → 反思 → 修复 √

每一步都产生**结构化 trace**：`trace = (state, action, observation, ρ)`

包含代码、命令、日志、工具输出、子 Agent 结果

——信息密度远超对话

用户反馈

唯一的确定性评判 — 不靠模型自评

- √ 成功 R=+1
- ? 追问 R=-0.3
- ✗ 纠正 R=-0.7

双层反馈机制

- **步级**：每步自动反思（根因定位 or 盲目试错？）
- **任务级**：用户对整轮结果判断 → 终局奖励锚点

效用量化

终局奖励 R → 反思加权回溯 → 每步 V

$$V(t) = \alpha \cdot R + (1-\alpha) \cdot \gamma \cdot V(t+1)$$

α 由反思质量决定 → 关键发现 α=0.7 | 常规推进 α=0.4 | 盲试 α=0.1

高 V 步骤 → 未来减少弯路的关键起点

Value Backfill

哪些步骤最有价值?

最有价值的不是最终结果，而是中间的根因定位步骤 — 复用它能避免未来弯路

价值 V 驱动四大下游

检索排序	策略归纳	决策修复	Skill 结晶
高 V trace 优先召回，减少重复试错	V 加权诱导跨任务 L2 subtask policy	高低 V 差异翻译成 anti-pattern	持续正增益 G>0 才升格为 Active Skill

每一步都有结构化证据，每个反馈都产生价值标签 — 这才是 Agent 记忆的核心资产。

执行时的记忆调用：Tier 1 / 2 / 3 分层检索

Skill 给骨架	Trace 补细节	World Model 定方向

Tier 1 · Skill　任务级检索

快速给出可执行策略骨架

成熟模式优先命中，直接注入 procedure、scope、verification。

命中示例

Skill "容器系统库缺失排障策略"

→ 注入 procedure + scope + anti-pattern

Tier 2 · Trace/Episode　步级检索

把失败步补成可复用操作细节

在 edge case 和新领域里，按 V 排序回填 command、patch、fallback。

命中示例

历史 trace "Alpine + lxml: apk add libxml2-dev" V=0.8 → 注入具体命令参考

Tier 3·场域认知　推理级检索

在不确定场景下直接判断方向

做根因定位、结构推理和环境预判，给出 diagnosis、rule、route。

命中示例

L3 "Python 包三层依赖"

→ Agent 预判当前包属哪一层

Tier 1　触发匹配

快速给出可执行策略骨架

查询条件：trigger　scope　eta

query+context → trigger match → Skill

命中后直接注入 procedure/verification/scope

Tier 2　相似经验检索

再找高价值相似经验

检索机制：error signature　embedding　V排序

当前失败步 → 结构匹配·语义匹配（标签过滤·相似经验聚合） → Top-K

按 V 值排序，回填最有用的命令与 patch

Tier 3　场域认知检索

最后用认知模型判断方向

触发场景：根因定位　结构推理　环境预判

深层反思 → 领域关键词·结构规则/约束禁忌·场域知识调用 → 导航

直接预判环境与约束，给出执行方向

完整执行例子 Docker 依赖安装

Skill 命中
命中 "容器系统库缺失排障策略"

Trace 补充
召回 apk add libxml2-dev 等高 V 经验

Model 预判
注入Python包三层依赖认知，判断当前包所属层

直接执行
Agent 无需试错，直接采用正确路径

我们的方案：Reflect2Evolve

交互驱动的智能体自我进化框架 — **双层反馈 × 三层认知 × 五层加工 × 能力结晶 × 分层检索** | 论文：MSCE（Memory - Skill Co-Evolution, training-free） · arXiv 2607.16621，EMNLP

MEMORY PRODUCTION 任务执行加工成长期能力

任务执行与反馈
把任务过程变成可学习证据

输入目标 → 调用工具 → 形成反思

目标 | 动作 | 反馈 | 反思

重点：记录真实动作

Episode 边界：纠正=续本轮 · 追问=新单元

五层记忆加工流水线
把一次执行加工成证据、策略、片段和世界模型

T	V	I	E	A
Trace	Value	Induce	Episode	Abstract
拆出步骤证据	回填成功 / 失败价值	跨任务归纳策略	拼接子任务片段	抽象环境规律
动作 / 观测 / 反思	关键步骤优先	trigger /procedure	检索时直接召回	结构 / 规律 / 约束

证据沉淀 | 策略归纳 | 模型抽象与技能结晶

沉淀记忆资产
三层治理记忆 + 可调用技能库

- **L1 原始记忆** (s, a, o, ρ, V) — 证据锚点
- **L2 操作手册** (ϕ, π, κ, B) + G — 可迁移策略
- **L3 环境地图** (E, I, C) — 声明式环境认知
- **Skill 调度能力** + A · D · η — 治理式升格 → 下页

反馈回流

MEMORY CORE 统一索引、分层存储、按需检索、动态更新

L1 · trace | L2 · policy | L3 · world model | Skill

MEMORY CONSUMPTION 按场景检索并注入 Agent

新任务
目标 + 约束 触发检索

记忆检索与注入
先找 Skill，再补 Trace / Episode / L3

Skill | Trace | Episode | L3

Agent 决策与执行
规划、执行、在线修正

规划 | 调用 | 修正

产出结果
回答 / 文件 / 命令

在线进化

输入：本轮轨迹 + 步级反思 α + 终局反馈 R

更新：回填 V → L2 → L3 → Skill 结晶

重点：不是重新训练，而是持续回写长期记忆。

从记忆到技能：Skill 结晶的治理式升格

MSCE · Memory–Skill Co-Evolution · training-free | arXiv 2607.16621

Skill 不是从嘈杂轨迹里蒸馏出来的，而是从治理过的记忆里升格出来的

1 L1 证据

步级 trace（s, a, o, ρ）

终局反馈回填价值 V

2 L2 策略

≥ 2 个 Episode 支撑才归纳

触发 · 流程 · 校验 · 边界

3 双门控

① 有证据且增益 G > 0

② 触发 / 流程 / 边界稳定

4 Skill 卡

确定性校验通过才可调用

schema · 证据 · 工具白名单

可靠性 η：平滑成功率

$$\eta = \frac{n_{\text{pass}} + 1}{n_{\text{trial}} + 2}$$

驱动生命周期：

试用 → 激活 · η ≥ 0.6 → 归档 · η < 0.2

策略增益 G：升格的入场券

$$G = \bar{V}_{\text{with}} - \bar{V}_{\text{without}}$$

有增益才可升格；持续负增益 → 退役重建

与策略相连的 trace 价值均值 − 其余 trace 均值

去掉 Skill 结晶会怎样

−6.2 ~ −11.5

五个领域 Pass@1 全部下降 —— 消融中最一致的退化

扁平记忆更差：SE −19.2 · Math −16.0

Training-free 只回写外部认知状态 C = (L1, L2, L3, Skill)，不更新基座模型 —— 记忆与技能共同进化

角色状态演化建模：PHASE-Tree

Psychology-grounded Hierarchical Attribute-Structured Evolving Tree | arXiv 2608.06975

角色的声音没忘，忘的是「角色已经变了」—— stale-state failure，状态过期失效

以《老友记》Chandler 为例

S1 · 讽刺，恐惧承诺

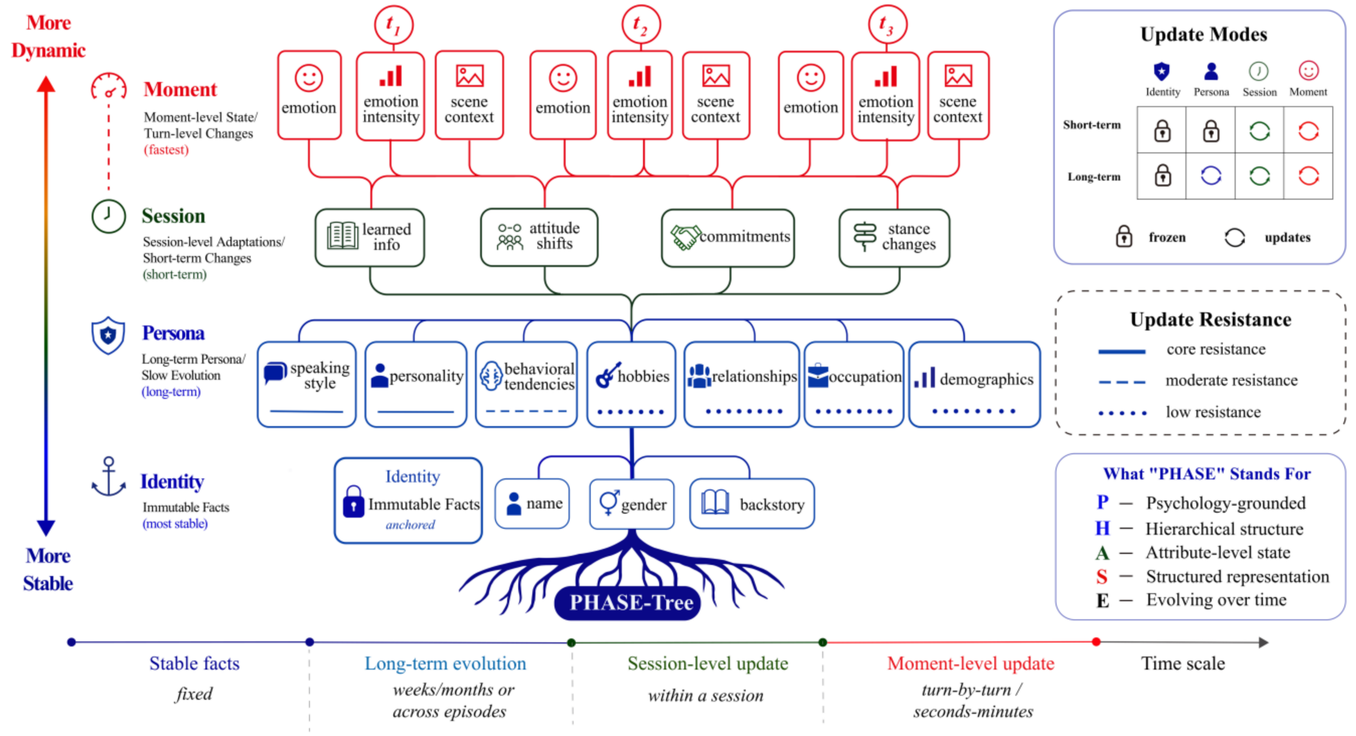
↓

S10 · 信赖伴侣的丈夫

婚礼场景里，静态画像的模型仍把「承诺」当笑点：**声音像，状态却停在过去。**

四层状态 · 阻力递增（右图）

- **Moment** 瞬时层 · 低阻力 · 每场刷新
- **Session** 会话层 · 中阻力 · 场内累积
- **Persona** 人格层 · 高阻力 · 跨集演化
- **Identity** 身份根 · 不可变



PHASE-Tree 角色状态层级（论文 Figure 1）：实线 = core、虚线 = moderate、点线 = low 阻力；右上为四层的短期 / 长期更新模式

状态更新 $S_{t+1} = U(S_t, E(c_t, y_t, S_t))$ 只改可变层，身份 | 永不变

PHASE-Tree 的更新机制

Intra-scene tracking × Cross-episode persona evolution | 阈值一次设定，四个长对话语料共用

场内看前缀，跨集看证据 —— 核心特质要跨越大半季的证据才会动

① 场内状态追踪

1 看前缀

只看已观察的对话前缀 + 场初画像，不偷看当前回复

2 抽状态

第三人称 session 条目 + moment 情绪快照

3 条件化

用它们生成本轮回复；证据进入下一目标的状态

② 跨集人格演化 · 阻力门控

$$\text{update}(f) \iff n_{ep} \geq \tau^{ep} \wedge n_{high} \geq \tau^{high} \wedge \Delta_{ep} \geq \tau^{cd}$$

广度 · 强度 · 冷却三重条件同时成立，才更新一个字段；每个字段独立判定

core	moderate	low
16 / 6 / 16	3 / 0 / 3	1 / 0 / 2
跨集数 / 高显著事件 / 冷却集数	跨集数 / 高显著事件 / 冷却集数	跨集数 / 高显著事件 / 冷却集数
性格 · 说话风格	**行为倾向**	**关系 · 职业 · 爱好 · 人口属性**
≈ 跨越 2/3 个电视季的证据	要求证据方向一致	1 个 high 或 2 个 medium 即可

合并：增量合并保留旧值；替换只用于明确矛盾（「男友」→「前男友」）

人工审核 0% 无据更新 · 77.6% 完全有据 · 22.4% 部分有据（152 条被接受的字段更新，三人审核）

LongEvoRoleBench：评测「以当前演化状态说话」

8 个角色扮演语料 · 统一 next-utterance 协议 · random + OOD 双留出

第一次把「以演化后的状态说话」当成与人设保持、记忆召回并列的一级任务

8 个语料 · 同一套协议

长对话 ×4 · 跨集演化核心测试

- Friends
- The Office
- Harry Potter
- Star Trek TNG

短对话 ×4 · 场内追踪对照

- RAIDEN
- CharacterEval
- SimsConv
- ChatHaruhi

长对话各追踪 6 位主角，关系 / 信念 / 情绪跨季演化；短对话画像静态，只测场内追踪。

评什么：角色的下一句

画像 S_t · time-t 的角色状态

+

场景上下文 c_t · 已观察的对话前缀

↓

角色的下一句 y_t

$$\hat{y}_t = \arg\max_y p_\theta(y \mid \text{flatten}(S_t), c_t)$$

评分对齐 time-t 的状态，而不是冻结画像；三种评分同时看

三指标 · 1–5 分

Char 人设一致
只依据画像文本评分

Sem 上下文连贯
只依据对话上下文评分

Emb 嵌入相似
与真值回复的余弦相似度

切分

- Random：留出早季与主流角色 → 测熟悉区间内的插值
- OOD：留出后期季 / 离群角色 → 测跨叙事时间的外推

结果：长对话 12 / 12 全部第一

显式文本注入 · Qwen2.5-7B 共享骨干 · random + OOD 均值 · 外部基线同骨干同评测管线

相对最强文本基线：Char +19.7% · Sem +12.4% · Emb +15.1%，且跨 4 个骨干、多种 judge 都成立

+19.7%
Char 人设一致
3.00 vs PAG 2.51

+12.4%
Sem 上下文连贯
3.70 vs RAG 3.29

+15.1%
Emb 嵌入相似
0.31 vs RAG 0.27

长对话均值（1–5 分）

关键发现

4 / 4
骨干（0.6B → 32B）上长对话 Sem 均第一

r = 0.65
200 条盲评：人评与 GPT-4.1 判分一致

DT → PT
唯一三指标齐升的转变：场内追踪是关键

落到 MemOS 属性记忆的算法内核：角色个性随经历自然生长 → NPC · 家居 · 座舱

开源 github.com/MemTensor/PHASE-Tree

GOSIM

GOSIM Shenzhen 2026

04

下一代记忆系统: MemOS x Metis 的统一记忆架构

大模型知道很多，但并不真正“记住”

信息可以进入上下文，却难以沉淀为长期状态，参与推理并持续进化。

KNOW

知道

预训练参数
承载大量世界知识

上下文窗口
处理当前任务信息

RAG与工具调用
检索和使用外部能力

Native Memory

原生记忆缺失

REMEMBER

记住

长期状态
保持用户、任务、环境的连续状态

记忆操作
写入、更新、删除、合并与冲突处理

持续进化
从交互中学习，不断改进行为

{ 从“知道”到“记住”，是大模型走向长期智能的关键跃迁。 }

记忆原生路径的迭代方向

记忆能力正在从运行时增强，走向模型内生的能力跃迁。

01

看到更多信息

Context-Augmented AI

Prompt | 上下文 | RAG | 工具调用

让模型在**一次推理**中看到更多信息和知识。

02

管理长期记忆

Memory-Augmented AI

MemOS | 记忆存储 | 生命周期 | 治理与审计

让记忆成为**可管理、可追溯、可治理**的系统对象。

03

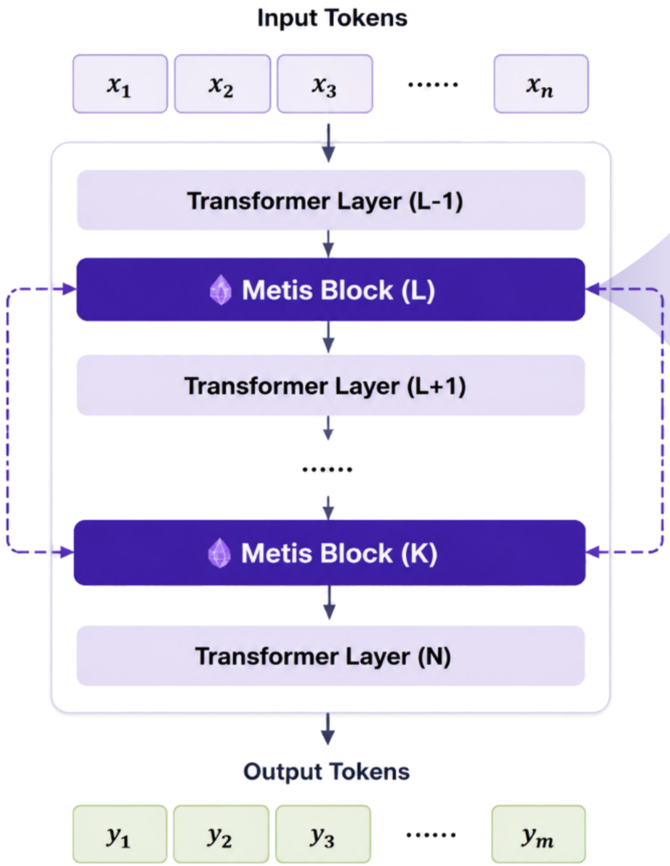
原生具备记忆能力

Memory-Native AI

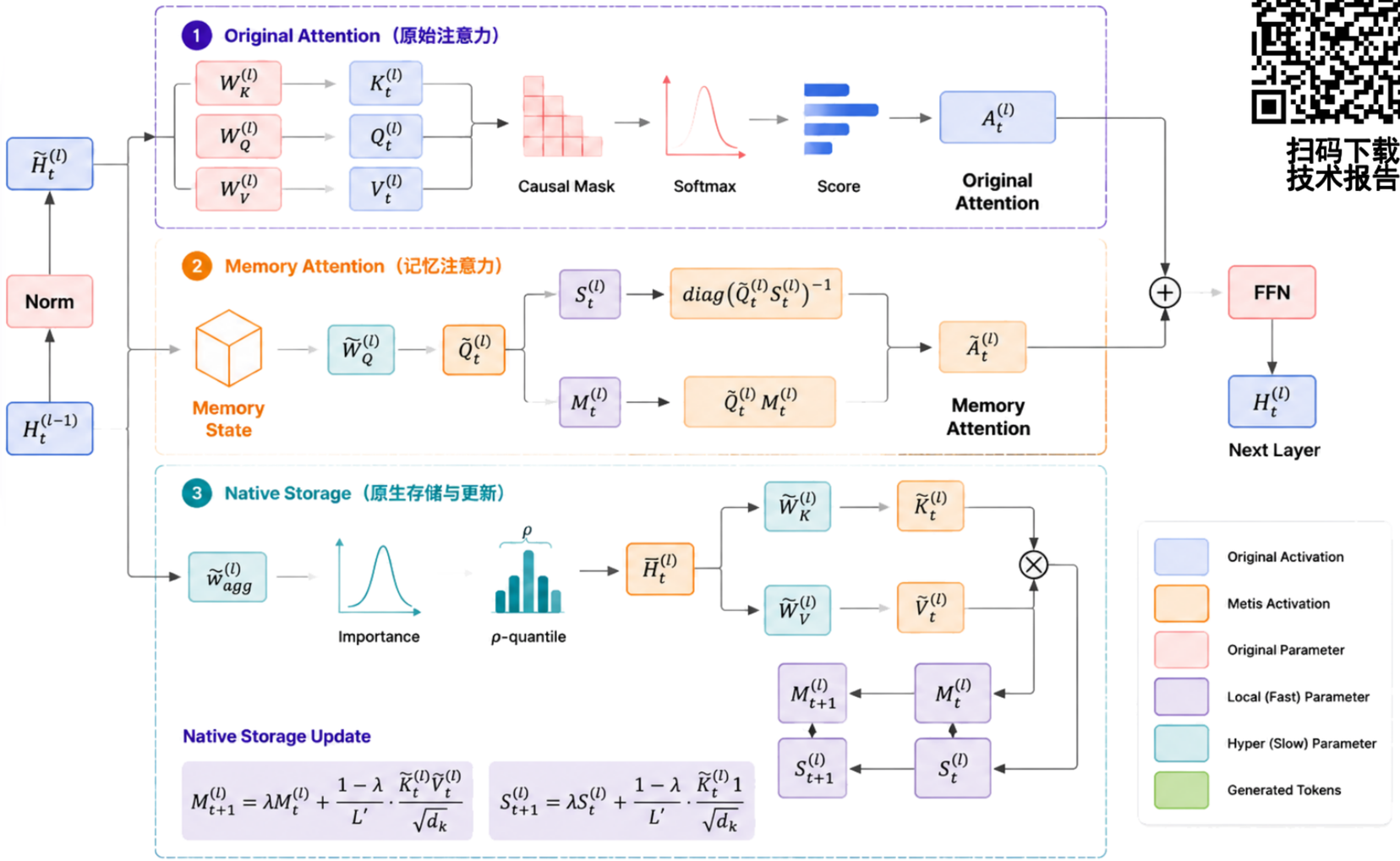
记忆表征 | 记忆操作 | 参与推理 | 持续进化

让记忆成为模型**内生能力**，天然具备对于记忆的良好操纵能力

两类记忆协同进化：MemOS 为原生记忆模型提供基础设施与治理能力。

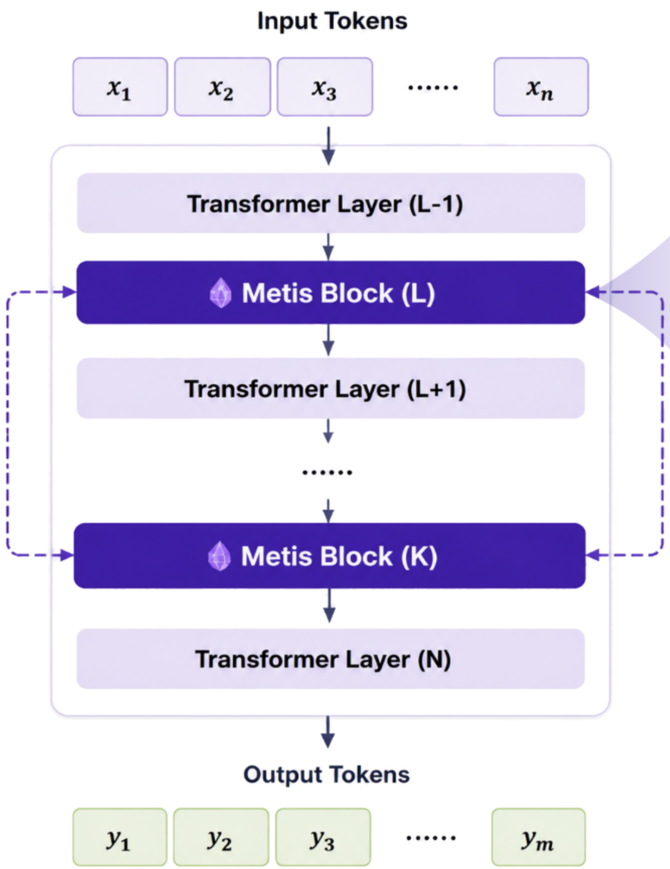


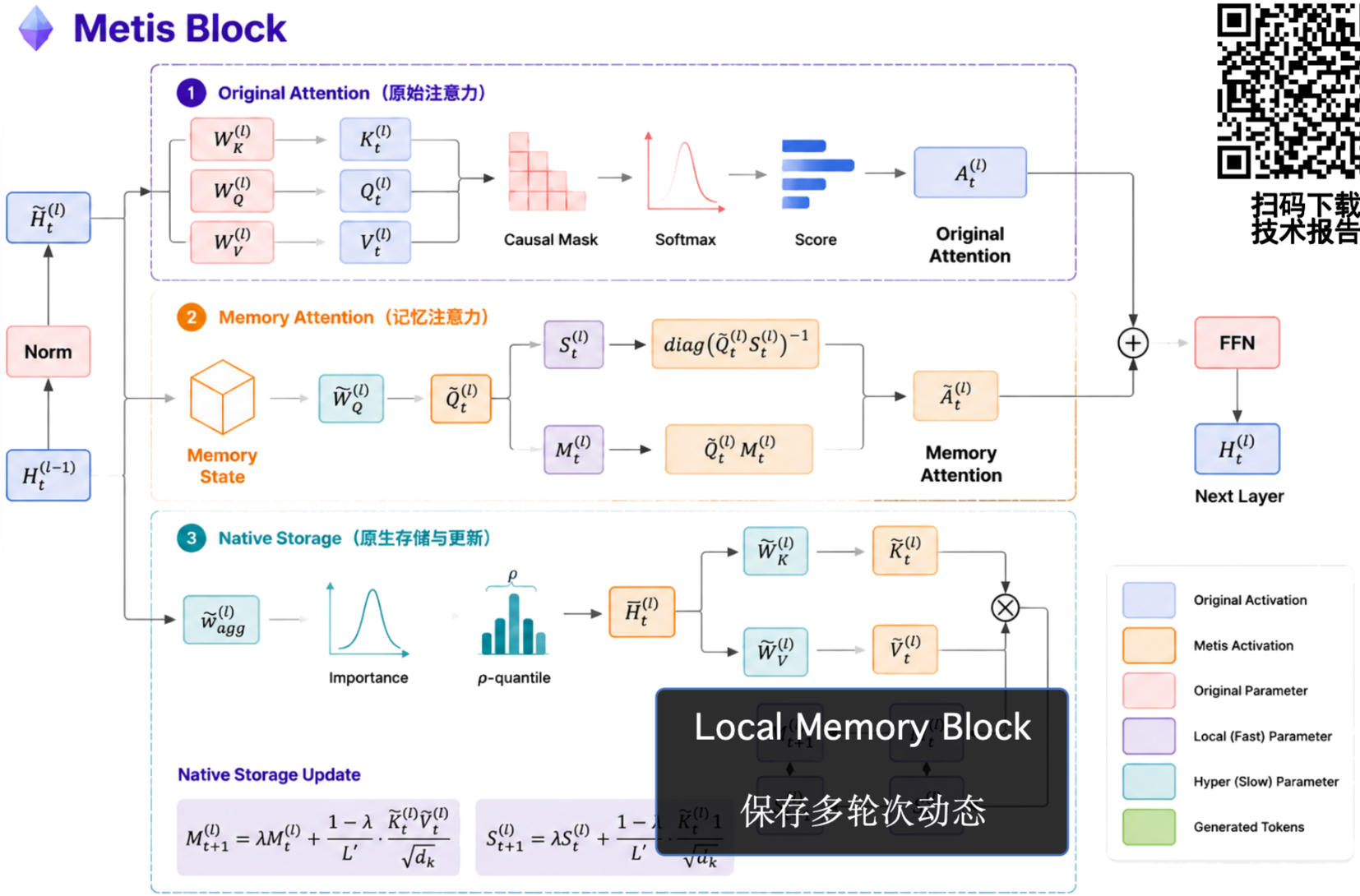
Metis Block



扫码下载
技术报告

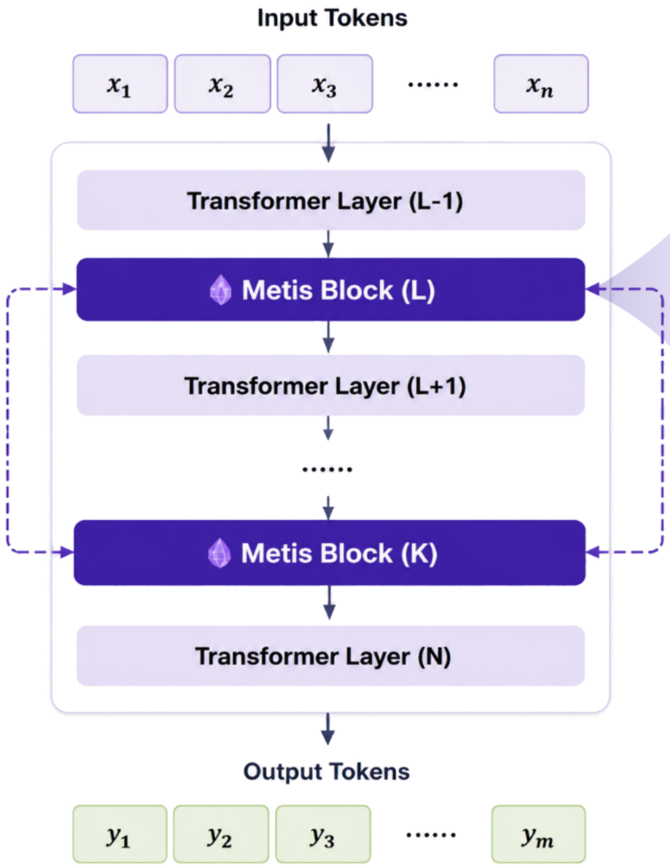
Metis：让模型具备原生记忆能力



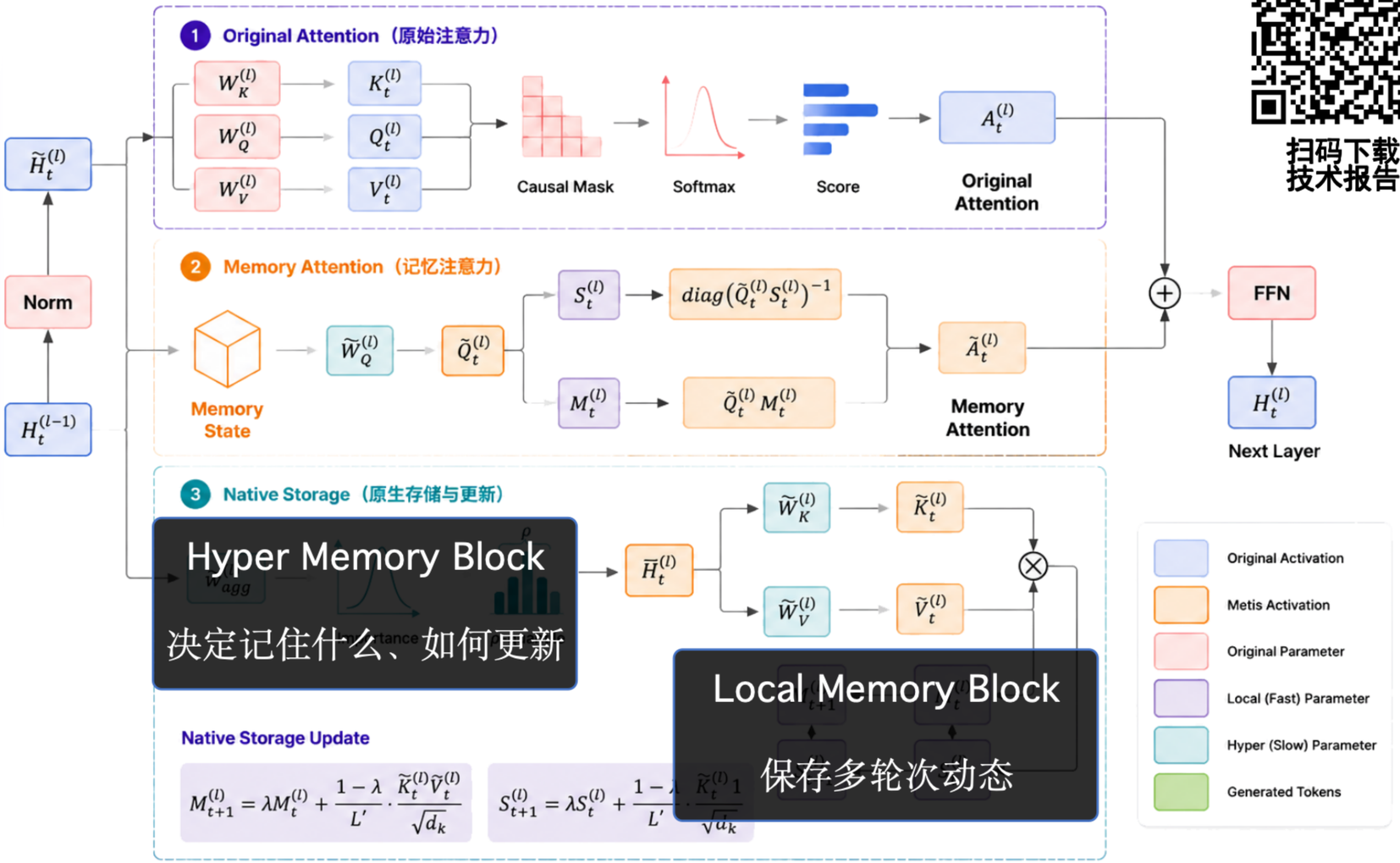


扫码下载技术报告

Metis：让模型具备原生记忆能力

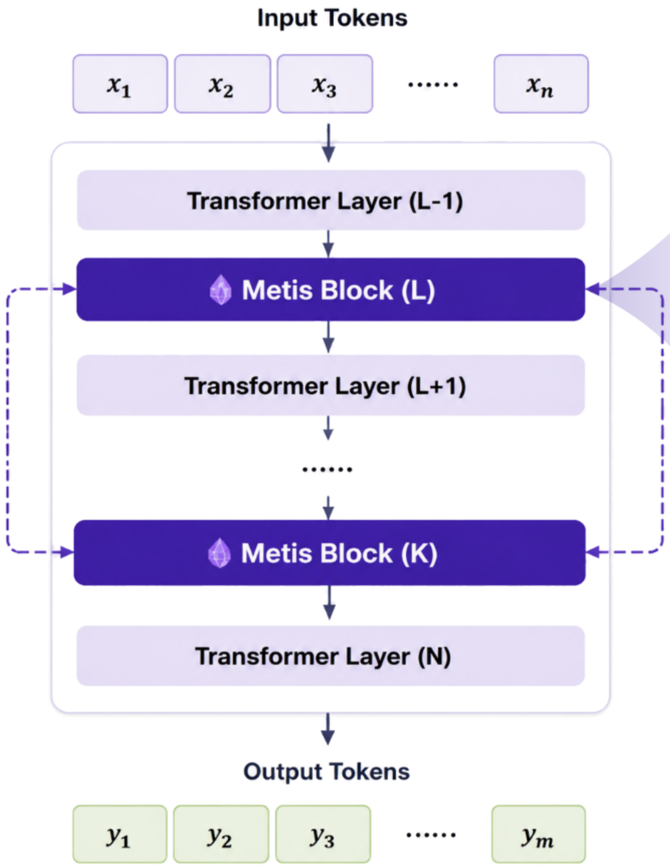


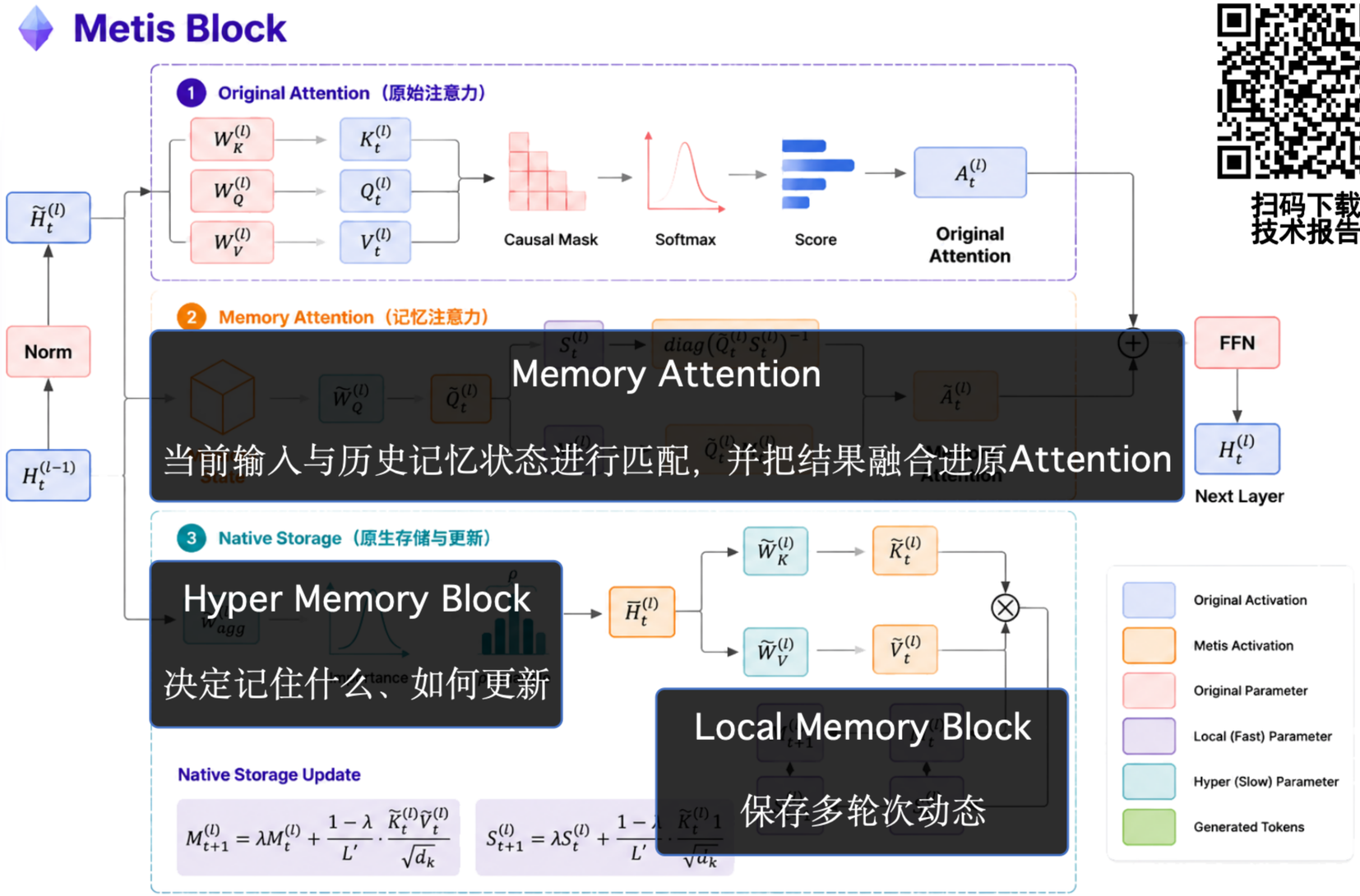
Metis Block



扫码下载技术报告

Metis：让模型具备原生记忆能力





扫码下载技术报告

MemOS x Metis 的统一记忆架构

Metis | 原生记忆操作

内生状态 · 原生过程 · 无梯度更新

本轮输入　问题 · 指令 · 上下文　→　隐藏状态 H_t　　中期训练：记忆重构 · 记忆操作 · 正则化

1 记什么　2 如何更新

写入通路 · 不承担审计

Hyper Memory Block

重要性评分 → Top-ρ 选择

K / V 投影 → GDU 更新

Local Memory Block

M_t 当前热状态
Alice 住址 = 北京

S_t 归一化与寻址
供 Memory Attention 使用

本轮读取：Memory Attention 读取 M_t、S_t
前向更新：M_{t+1}、S_{t+1} → 下轮使用

按 key 局部修正

- 以当前 key 为条件
- 目标是局部修正
- 减少无关事实干扰
- 热缓存不作核对依据

提交请求

ADD · UPDATE · FORGET

- 内容 content
- 作用域 scope
- commit(op, content, scope)

3 如何调用

Alice 住址回溯 · Query → GET → 伦敦记录 → 融合

1 Query 投影

问题：Alice 之前住在哪里?

ToolCall · 精确获取
GET(Alice, 搬来前, 住址, 上一版)

从“之前”解析时间关系
目标：返回 v1 的住址记录

当前 M_t 已编码：北京

2 返回 MemCube

外部可解释事实

v1 · 搬来前 · 住址 · 伦敦

进入本轮上下文
交给原始 Attention

来源：核验后的历史记录
北京（v2）不在本次返回

3 并行融合后推理

Memory Attention
内生读出：北京 · 现在

原始 Attention
本轮上下文：伦敦 · v1

结合时间关系推理

回答：伦敦

装入 M_t　　提交请求　　GET　　返回这一条

MemOS | 记忆基础设施

保存 · 校验 · 返回 · 装入

$M+S$ 快照 当前热状态已编码：北京

user_id × model_version

- 固定大小 · 保存当前压缩状态
- 装入为各层 M_t、S_t · 不保存历史原话

写入与治理

- 权限先核验，再返回 v1
- GET 读取不改写 MemCube
- 前向后更新下一轮 M、S

MemCube | 历史库

GET → 精确返回 v1：伦敦

事实内容 · payload	版本与来源	治理边界
Alice · 住址 · 伦敦	v1 · 搬来前 · 核验记录	user: Alice · ttl · permission

MemCube = Alice 搬来北京之前的住址事实；本次 GET 只返回 v1 伦敦，当前热状态仍编码北京。

统一记忆架构：从经验存储到自主智能

记忆进入模型运行回路 · 能力形成自我改进闭环 · 企业经验沉淀为自主智能资产

01

记忆进入模型运行回路

经验成为模型状态

写入模型状态　推理直接调用　新增无需重训

× 外挂记忆：经验停留在 Prompt
每次调用都要重新检索、拼接和解释。

✓ 记忆原生：经验写入模型状态
新增经验无需重训主干，可直接参与推理。

02

能力形成自我改进闭环

能力提升进入闭环

诊断能力短板　归因失败原因　生成定向数据

× 数据堆积：训练很难找到短板
任务失败难以转成下一轮训练数据。

✓ 能力闭环：短板直接进入训练
诊断、归因、生成和验证连续衔接。

03

企业经验沉淀为自主智能资产

企业经验成为智能资产

沉淀业务知识　复用实践经验　能力持续演进

× 外部调用：经验散落在项目之外
业务知识和实践成果难以沉淀为企业能力。

✓ 自主智能：经验成为可继承资产
知识、实践和改进成果随业务持续复用。

让经验进入模型、进入训练闭环、进入企业能力资产。

GOSIM

GOSIM Shenzhen 2026

Thanks!

GOSIM®
GLOBAL OPEN-SOURCE INNOVATION MEETUP

GOSIM Shenzhen 2026

智能体操作系统与应用

Agentic OS & App